

TUBO – FULL-STACK DEVELOPER TECHNICAL ASSESSMENT

Candidate:

Position: Full-Stack Developer

Assessment Type: Practical Development + Architecture + Technical Reasoning

1. OBJECTIVE

This assessment evaluates your ability to design and implement a production-oriented full-stack application.

The test focuses on:

- Frontend development
- Backend/API development
- Database design
- Authentication and security
- Transaction handling
- Error handling
- Asynchronous processing
- Retry mechanisms
- Idempotency
- Scalability
- Code quality
- Technical decision-making

You are not expected to build a complete production system within the available time. We are interested in your **engineering approach, working implementation, and ability to explain your decisions.**

PART A – PRACTICAL DEVELOPMENT

Scenario

You are working on an electronic invoicing platform called **Tubo**.

Businesses send invoices to Tubo. Tubo validates and stores the invoices and then submits them to an external government invoicing service.

The external service may occasionally:

- respond slowly;
- become temporarily unavailable;
- reject invalid invoices;
- return an error;
- successfully receive an invoice while Tubo experiences an internal failure.

Your system must therefore be designed so that invoices are not accidentally lost or duplicated.

PART B – DATA MODEL

An invoice should contain at least the following information:

Invoice Header

- Invoice Number
- Invoice Date
- Seller/Company
- Customer Name
- Customer Tax ID
- Customer Email
- Currency
- Subtotal
- Tax Amount
- Total Amount
- Invoice Status
- Created Date
- Updated Date

Invoice Items

Each invoice can contain multiple items.

Each item should contain:

- Description
- Quantity
- Unit Price
- Tax
- Line Total

You may introduce additional fields that you believe are necessary.

PART C – DATABASE DESIGN

Design an appropriate relational database structure.

At minimum, consider the following entities:

- Users
- Companies
- Invoices
- Invoice Items
- Processing Logs

You are responsible for deciding:

- Primary keys
- Foreign keys
- Relationships
- Data types
- Unique constraints
- Indexes
- Status representation

Question C1

Explain why you designed the database this way.

Question C2

How will your database prevent the same company from accidentally creating the same invoice twice?

Question C3

Assume two identical requests arrive at exactly the same time.

Both requests contain:

Company: ABC Corporation

Invoice Number: INV-10001

How will you guarantee that only one invoice is created?

PART D – BACKEND API

Develop REST APIs supporting the following operations.

1. Create Invoice

POST /api/invoices

The API should:

- authenticate the requester;
 - validate the request;
 - validate invoice items;
 - calculate or validate totals;
 - detect duplicates;
 - store the invoice;
 - return an appropriate HTTP response.
-

2. List Invoices

GET /api/invoices

Support reasonable filtering such as:

- status;
- invoice date;
- invoice number.

Pagination is recommended.

3. Get Invoice

GET /api/invoices/{id}

Return the invoice header, items and relevant processing information.

4. Retry Failed Invoice

`POST /api/invoices/{id}/retry`

Only invoices in an appropriate failed/retryable state should be eligible for retry.

PART E – AUTHENTICATION & SECURITY

Implement a basic authentication mechanism using JWT or another appropriate approach.

Assume:

Company A has User A.

Company B has User B.

Question E1

How will you guarantee that User A cannot access Company B's invoices by manually changing an invoice ID in the URL?

Question E2

Where should the following be stored?

- Database passwords
- JWT secrets
- External API credentials

Explain what should **not** be committed to Git.

Question E3

What security checks would you implement before accepting an invoice API request?

PART F – FRONTEND

Create a simple frontend using **React** or **Next.js**.

The frontend should contain at least:

Invoice Dashboard

Display:

- Invoice Number
- Customer
- Invoice Date
- Amount
- Status

Suggested statuses:

PENDING

PROCESSING

SUBMITTED

FAILED

Create Invoice

Provide a form where a user can:

- enter invoice information;
- add multiple invoice items;
- remove invoice items;
- submit the invoice;
- see validation errors.

Invoice Details

Display:

- invoice information;
- invoice items;
- current status;
- processing/error information.

For a failed invoice, provide a **Retry** action where appropriate.

The UI does not need to be visually sophisticated. Correct functionality, maintainability and usability are more important.

PART G – EXTERNAL SERVICE INTEGRATION

Assume Tubo must send each invoice to:

`POST /government-api/invoices`

For the purpose of this assessment, you may mock this service.

The mock service can randomly return:

Success

HTTP `200`

Temporary Failure

HTTP `503`

Invalid Invoice

HTTP `400`

Timeout

No response within the expected period.

Design your application so these situations are handled appropriately.

PART H – ASYNCHRONOUS PROCESSING

The API should ideally **not wait for the external government service to finish processing before accepting the invoice.**

Example:

Client

→ Tubo API

→ Database

- Processing mechanism
- External Government API

Question H1

How would you implement this asynchronously?

You may implement or describe the use of:

- Job queues
- Workers
- Background jobs
- Message queues

Question H2

Explain the difference between accepting an invoice and successfully submitting it to the external system.

Question H3

What happens if the external API remains unavailable for 30 minutes?

Design an appropriate retry strategy.

PART I – CRITICAL FAILURE SCENARIO

This is an important part of the assessment.

Consider this sequence:

1. Turbo receives Invoice **INV-10001**.
2. Turbo stores it.
3. A worker sends it to the external API.
4. The external API successfully accepts the invoice.
5. Immediately afterward, the Turbo server crashes.
6. Turbo therefore does not update its database to **SUBMITTED**.
7. The server restarts.
8. The worker sees the invoice as incomplete.

Question I1

What should happen next?

Question I2

How will you prevent Turbo from creating a duplicate submission in the external system?

Question I3

Explain **idempotency** and how you would use it in this scenario.

PART J – CONCURRENCY PROBLEM

Review the following pseudo-code:

```
const existingInvoice = await Invoice.findOne({
  companyId: companyId,
  invoiceNumber: invoiceNumber
});

if (!existingInvoice) {

  await Invoice.create({
    companyId,
    invoiceNumber,
    total
  });

}
```

Two requests for **INV-10001** arrive at almost exactly the same time.

Questions

1. Is the code safe?
2. What could happen?
3. Why could both requests pass the check?
4. How would you fix it?
5. Would application-level validation alone be sufficient?
6. What database mechanism would you use?

You may modify the code or explain your solution.

PART K – SCALABILITY

Assume Tubo currently processes:

10,000 invoices/day

Six months later it needs to process:

1,000,000 invoices/day

You cannot simply assume that one server can process everything.

Question K1

Explain how you would modify the architecture.

Consider:

- API servers
- Load balancing
- Database
- Queue
- Workers
- Caching
- File/object storage
- Logging
- Monitoring

Question K2

Suppose 100,000 invoices arrive within 10 minutes.

The external government API only allows:

100 requests/second

How would you prevent Tubo from overwhelming the external API?

Question K3

Would you process all invoices simultaneously?

Explain your approach.

PART L – FAILURE & RETRY DESIGN

Assume the following invoice:

INV-20001

has failed three times.

Design how Turbo should record each attempt.

We should be able to determine:

- when processing started;
- when it ended;
- attempt number;
- HTTP response;
- error message;
- processing status.

Question L1

Would you overwrite the previous error every time?

Explain.

Question L2

How would a support engineer investigate why an invoice failed three days ago?

PART M – API VERSIONING

Turbo currently exposes:

POST /api/v1/invoices

Six months later, a major customer requires additional fields and the payload structure needs to change.

Question

How would you introduce the change without immediately breaking existing customers using V1?

Explain your API versioning strategy.

PART N – DEBUGGING SCENARIO

Production monitoring shows:

- CPU: 35%
- Memory: 55%
- Database CPU: 20%
- API response time: normal
- Queue waiting jobs: 150,000
- Workers: 5
- External API: responding normally

Customers complain that invoices take several hours to process.

Questions

1. Where would you investigate first?
 2. What do these metrics suggest?
 3. What additional metrics would you check?
 4. What changes might improve throughput?
 5. How would you ensure increasing worker count does not overload another component?
-

PART O – CODE QUALITY

Your submission should demonstrate reasonable:

- Project structure
- Naming conventions
- Separation of concerns
- Error handling
- Input validation
- Reusable code
- Database design
- Security practices
- Git usage

Do not spend excessive time making the UI visually perfect.

We are more interested in **how the application works and how it has been engineered**.

PART P – README

Include a README containing:

Setup

Explain how to run:

- Frontend
- Backend
- Database

Architecture

Briefly explain:

- application structure;
- database design;
- invoice processing flow;
- authentication approach.

Assumptions

Document any assumptions you made.

Production Improvements

List the improvements you would make if this application were going to production.

PART Q – TECHNICAL DISCUSSION

After submission, you will have a technical discussion with the interview panel.

Be prepared to explain your own code and answer questions such as:

1. Why did you choose this architecture?
2. What is the weakest part of your current implementation?
3. What would you improve with another two days?
4. What happens when the database becomes unavailable?
5. What happens if a worker crashes?
6. How do you prevent duplicate processing?
7. How do you scale workers?
8. How do you protect one company's data from another company?

9. How would you monitor the application in production?

10. How would you deploy a new version without significantly interrupting invoice processing?

SUBMISSION REQUIREMENTS

Submit:

1. Source code
2. Database schema/migrations
3. README
4. API documentation or Postman collection
5. Any architecture diagram you believe is useful

Git repository submission is preferred.

USE OF AI TOOLS

You may use AI-assisted development tools.

However, you must be able to:

- explain every major part of your solution;
- modify your implementation during the technical discussion;
- debug your own code;
- justify architectural decisions;
- explain code generated with AI assistance.

Code that works but cannot be explained by the candidate will not be considered sufficient.

IMPORTANT

This assessment is not about completing the maximum number of features.

Priority should be:

Correctness → **Data Integrity** → **Backend Design** → **Reliability** → **Security** → **Maintainability** → **UI**

Where time is limited, implement the core functionality and clearly explain how you would complete the remaining production requirements.